# beimingh

September 4, 2026

## 1 Collaborators

List your collaborators here:

---

### 1.1 Initialization

Run the following code to import the modules you'll need. After your finish the assignment, remember to run all cells and save the note book to your local machine as a PDF for gradescope submission.

[ ]:
```
import math
import matplotlib.pyplot as plt
import numpy as np
import os
import platform
import random
from random import randrange
import time
import torch
import torchvision
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import torchvision.transforms as transforms
import glob
from skimage.util import montage

np.random.seed(0)
```

## 2 1. Setup dataset

In this section we will download the dataset, unzip it and setup the paths to load images from.

This dataset is a tiny subset of ImageNet, a popular dataset for image classification.

This tiny dataset has **9538 training** images and **3856 test** images spanning **10 classes** {*fish, English-springer, cassette-player, chain-saw, church, French-horn, garbage-truck, gas-pump, golf-ball, parachute*} stored in the following directory structure:

```
dataset
  ---train
     ---class1
     ---class2
     ...
  ---test
     ---class1
     ---class2
     ...
```

The data has been cleaned and we have provided dataloading functions below so you can directly use the dataset.

[3]:
```
if not os.path.exists('imagenette'):
  #!wget "https://drive.google.com/uc?
 ↪export=download&id=1t3XtxcpVwZnKhsM95Q89MxYNlX5mj6aJ&confirm=t" -O /content/
 ↪imagenette.zip
  !wget https://www.cs.cmu.edu/~deva/data/imagenette.zip -O /content/imagenette.
 ↪zip
  !unzip -qq "/content/imagenette.zip"
```

```
--2026-09-03 18:09:22--  https://www.cs.cmu.edu/~deva/data/imagenette.zip
Resolving www.cs.cmu.edu (www.cs.cmu.edu)… 128.2.42.95
Connecting to www.cs.cmu.edu (www.cs.cmu.edu)|128.2.42.95|:443… connected.
HTTP request sent, awaiting response… 200 OK
Length: 107288983 (102M) [application/zip]
Saving to: ‘/content/imagenette.zip’

/content/imagenette 100%[===================>] 102.32M  1.33MB/s    in 81s

2026-09-03 18:10:43 (1.27 MB/s) - ‘/content/imagenette.zip’ saved
[107288983/107288983]
```

[ ]:
```
train_data_path = '/content/imagenette/train'
test_data_path = '/content/imagenette/test'
train_image_paths = [] #to store image paths in list
test_image_paths  = []
classes           = []

#Get all the paths from train_data_path and append image paths and class to to␣
 ↪respective lists

for data_path in glob.glob(train_data_path + '/*'):
    classes.append(data_path.split('/')[-1])
    train_image_paths.append(glob.glob(data_path + '/*'))

for data_path in glob.glob(test_data_path + '/*'):
```

```
    test_image_paths.append(glob.glob(data_path + '/*'))

train_image_paths = list(sum(train_image_paths,[]))
random.shuffle(train_image_paths)
test_image_paths = list(sum(test_image_paths,[]))
random.shuffle(test_image_paths)

idx_to_class = {i:j for i, j in enumerate(classes)}
class_to_idx = {value:key for key,value in idx_to_class.items()}
```

[ ]:
```
def LoadData(img_paths,img_size,class_to_idx):
  n = len(img_paths)
  Images = np.zeros((n,img_size,img_size,3),dtype='uint8')
  Labels = np.zeros(n)
  for i in range(n):
    path = img_paths[i]
    Images[i,:,:,:] = np.asarray(Image.open(path).resize((img_size,img_size)));
    Labels[i] = class_to_idx[path.split('/')[-2]]
  return Images, Labels

# Load images as size 32x32; you can try with img_size = 64 to check if it␣
 ↪improves the accuracy
img_size = 32
Train_Images, Train_Labels = LoadData(train_image_paths, img_size, class_to_idx)
Test_Images,   Test_Labels = LoadData( test_image_paths, img_size, class_to_idx)
```

[ ]:
```
# Visualize the first 5 images of the 10 classes
plt.figure(figsize=(15,15))
for i in range(10):
  plt.subplot(10,1,i+1)
  ind = np.nonzero(Train_Labels == i)[0]
  plt.imshow(montage(Train_Images[ind[:5],:],grid_shape=(1,5),channel_axis=3))
  plt.axis('off');
  plt.title(idx_to_class[i])
```

garbage-truck



chain-saw



fish



parachute



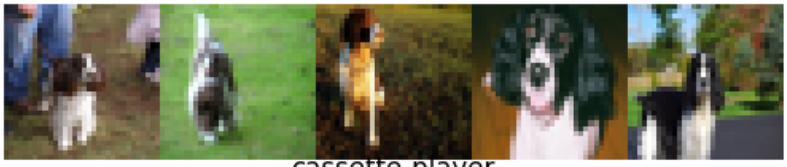
gas-pump



French-horn



English-springer



cassette-player



golf-ball



church



## 2.1 Debug Flag

Set the debug flag to true when testing. Setting the debug flag to true will let the dataloader use only 20% of the training dataset, which makes everything run faster. This will make testing the code easier.

Once you finish the coding part please make sure to change the flag to False and rerun all the cells. This will make the colab ready for submission.

[ ]:
```
DEBUG = False

# Take a smaller subset of the training set for efficient execution of kNN
# We also create a small validation set

if DEBUG:
  num_train = 1900
  num_test =  700
else:
  num_train = 9000
  num_test =  3856

X_train = Train_Images[:num_train].reshape(num_train,-1).astype('float64')
y_train = Train_Labels[:num_train]
X_test  = Test_Images[:num_test].reshape(num_test,-1).astype('float64')
y_test  = Test_Labels[:num_test]

print('Train data shape: '  , X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Test data shape: '   , X_test.shape)
print('Test labels shape: ' , y_test.shape)
```

```
Train data shape:  (9000, 3072)
Train labels shape:  (9000,)
Test data shape:  (3856, 3072)
Test labels shape:  (3856,)
```

## 2.2 Problem 3.1

### 2.2.1 (a) Define the KNearestNeighbor class

[ ]:
```
from collections import Counter
import numpy as np

class KNearestNeighbor(object):
  """ a kNN classifier with L2 distance """
```

```
def __init__(self):
  pass

def train(self, X, y):
  """
  Train the classifier. For k-nearest neighbors this is just
  memorizing the training data.
  Inputs:
  - X: A numpy array of shape (num_train, D) containing the training data
    consisting of num_train samples each of dimension D.
  - y: A numpy array of shape (N,) containing the training labels, where
       y[i] is the label for X[i].
  """
  self.X_train = X
  self.y_train = y

def predict(self, X, k=1, num_loops=0):
  """
  Predict labels for test data using this classifier.
  Inputs:
  - X: A numpy array of shape (num_test, D) containing test data consisting
       of num_test samples each of dimension D.
  - k: The number of nearest neighbors that vote for the predicted labels.
  - num_loops: Determines which implementation to use to compute distances
    between training points and testing points.
  Returns:
  - y: A numpy array of shape (num_test,) containing predicted labels for the
    test data, where y[i] is the predicted label for the test point X[i].
  """
  if num_loops == 0:
    dists = self.compute_distances_no_loops(X)
  elif num_loops == 1:
    dists = self.compute_distances_one_loop(X)
  elif num_loops == 2:
    dists = self.compute_distances_two_loops(X)
  else:
    raise ValueError('Invalid value %d for num_loops' % num_loops)

  return self.predict_labels(dists, k=k)

def compute_distances_two_loops(self, X):
  """
  Compute the l2 distance between each test point in X and each training point
  in self.X_train using a nested loop over both the training data and the
  test data.
  Inputs:
  - X: A numpy array of shape (num_test, D) containing test data.
```

```
    Returns:
    - dists: A numpy array of shape (num_test, num_train) where dists[i, j]
      is the Euclidean distance between the ith test point and the jth training
      point.
    """
    num_test = X.shape[0]
    num_train = self.X_train.shape[0]
    dists = np.zeros((num_test, num_train))
    for i in range(num_test):
      for j in range(num_train):
        dists[i,j]=np.linalg.norm(X[i]-self.X_train[j])
    return dists

  def compute_distances_one_loop(self, X):
    """
    Compute the l2 distance between each test point in X and each training point
    in self.X_train using a single loop over the test data.
    Input / Output: Same as compute_distances_two_loops
    """
    num_test = X.shape[0]
    num_train = self.X_train.shape[0]
    dists = np.zeros((num_test, num_train))
    for i in range(num_test):
      dists[i, :] = np.linalg.norm(X[i] - self.X_train, axis=1)
    return dists

  def compute_distances_no_loops(self, X):
    """
    Compute the l2 distance between each test point in X and each training point
    in self.X_train using no explicit loops.
    Input / Output: Same as compute_distances_two_loops
    """
    num_test = X.shape[0]
    num_train = self.X_train.shape[0]

    sum_sq_test = np.sum(X**2, axis=1, keepdims=True)

    sum_sq_train = np.sum(self.X_train**2, axis=1, keepdims=True).T

    dot_product = X @ self.X_train.T


    dists_sq = sum_sq_test + sum_sq_train - 2 * dot_product

    dists_sq = np.maximum(dists_sq, 0)

    dists = np.sqrt(dists_sq)
```

```
    return dists

  def predict_labels(self, dists, k=1):
    """
    Given a matrix of distances between test points and training points,
    predict a label for each test point.
    Inputs:
    - dists: A numpy array of shape (num_test, num_train) where dists[i, j]
      gives the distance betwen the ith test point and the jth training point.
    Returns:
    - y: A numpy array of shape (num_test,) containing predicted labels for the
      test data, where y[i] is the predicted label for the test point X[i].
    - knn_idxs: List of arrays, containing Indexes of the k nearest neighbors
      for the test data. So, for num_tests, it will be a list of length
      num_tests with each element of the list, an array of size 'k'. This will
      be used for visualization purposes later.
    """
    num_test = dists.shape[0]
    y_pred = np.zeros(num_test)
    knn_idxs = []
    for i in range(num_test):
      # A list of length k storing the labels of the k nearest neighbors to
      # the ith test point.
      indx=np.argsort(dists[i, :])[:k]
      closest_y = self.y_train[indx]
      knn_idxs.append(indx)

      vote = Counter(closest_y)
      count = vote.most_common()
      y_pred[i] = count[0][0]

    return y_pred, knn_idxs
```

### 2.2.2 (b) Check L2 distance implementation

Now, let's do some checks to see if you have implemented the functions correctly. We will first calculate distances using ***compute_distance_two_loops*** and check accuracy for k=1 and k=3. Then, we will compare the ***compute_distance_one_loop*** and ***compute_distance_no_loop*** with ***compute_distance_two_loops*** to ensure all results are consistent.

Initialize the KNN Classifier

[ ]:
```
classifier = KNearestNeighbor()
classifier.train(X_train,y_train)
```

Compute the distance between the training and test set. This might take some time to run since we are running the two loops function which is not efficient.

**6 to 8 mins for full dataset | 2 to 3 mins for debug dataset**

[ ]:
```
dists_two = classifier.compute_distances_two_loops(X_test)
```

Now, let's do some checks to see if you have implemented the functions correctly. We will first calculate the distances using compute_distance_two_loops function and check the accuracies for k=1 and k=3. Then, we will compare the compute_distance_one_loop and compute_distance_no_loop functions with it to check their correctness.

Predict labels and check accuracy for k = 1. You should expect to see approximately 28% accuracy for full dataset.
**(Accuracy below 24% on full dataset (Debug = False) will not be given full grades)**

[ ]:
```
y_test_pred, k_idxs  = classifier.predict_labels(dists_two, k=1)
# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 1077 / 3856 correct => accuracy: 0.279305
```

Now lets check the one loop implementation. This should also take some time to run.
**4 to 6 mins for full dataset | 1 to 2 mins for debug dataset**

**Note:** This function can possibly take a little more time that two loop implementaion because of some quirks in python, numpy and cpu processing. It is fine as long as the final output shows no difference below.

[ ]:
```
# Implement the function compute_distances_one_loop in KNearestNeighbor class
# and run the code below:
dists_one = classifier.compute_distances_one_loop(X_test)

# To ensure that our vectorized implementation is correct, we make sure that it
# agrees with the naive implementation. There are many ways to decide whether
# two matrices are similar; one of the simplest is the Frobenius norm. In case
# you haven't seen it before, the Frobenius norm of two matrices is the square
# root of the squared sum of differences of all elements; in other words,␣
 ↪reshape
# the matrices into vectors and compute the Euclidean distance between them.

difference = np.linalg.norm(dists_two - dists_one, ord='fro')
print('Difference was: %f' % (difference, ))
if difference < 0.001:
  print('Good! The distance matrices are the same')
else:
  print('Uh-oh! The distance matrices are different')
```

```
Difference was: 0.000000
Good! The distance matrices are the same
```

Now lets check the vectorized implementation. This should take less than 30 secs to run for full dataset.

[ ]:
```
# Now implement the fully vectorized version inside compute_distances_no_loops
# and run the code
dists_no = classifier.compute_distances_no_loops(X_test)
# check that the dist ance matrix agrees with the one we computed before:
difference = np.linalg.norm(dists_two - dists_no, ord='fro')
print('Difference was: %f' % (difference, ))
if difference < 0.001:
  print('Good! The distance matrices are the same')
else:
  print('Uh-oh! The distance matrices are different')
```

```
Difference was: 0.000000
Good! The distance matrices are the same
```

Let's compare how fast the implementations are You should see significantly faster performance with the fully vectorized implementation

[ ]:
```
def time_function(f, *args):
    """
    Call a function f with args and return the time (in seconds) that it took␣
 ↪to execute.
    """
    import time
    tic = time.time()
    f(*args)
    toc = time.time()
    return toc - tic

two_loop_time = time_function(classifier.compute_distances_two_loops,X_test)
print('Two loop version took %f seconds' % two_loop_time)

one_loop_time = time_function(classifier.compute_distances_one_loop,X_test)
print('One loop version took %f seconds' % one_loop_time)

no_loop_time = time_function(classifier.compute_distances_no_loops,X_test)
print('No loop version took %f seconds' % no_loop_time)

# you should see significantly faster performance with the fully vectorized␣
 ↪implementation
```

```
Two loop version took 351.066455 seconds
One loop version took 768.042809 seconds
No loop version took 7.167110 seconds
```

### 2.3 From this point on, we will use the efficient no loop implementation

The given accuracy of 29% is much better than chance accuracy of

## 3 ===== your answer here! =====

The given accuracy of 29% is much better than chance accuracy of guessing, which is 10%. Since we have 10 classes, and a random guess is correct 1/10 of the time. # ===== end of your answer =====

Though the no-loop impementation is far faster, there maybe situations where one_loop or two_loop implementations are useful, such as [HINT: Imagine really large training set and or testset]

## 4 ===== your answer here! =====

When the training set or the test set is too large to fit in the memory. Then 1 or 2 loops are useful for implementation. # ===== end of your answer =====

[ ]:
```
y_test_pred, k_idxs  = classifier.predict_labels(dists_no, k=3)
# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 1126 / 3856 correct => accuracy: 0.292012
```

### 4.0.1 Visualize KNN results

Let's visualize the K nearest images for some randomly selected examples from the test set using the k_idxs list you returned in predict_labels.

Here the leftmost column is the input image from the test set and rest of the columns are the K nearest neighbors from the training set

[ ]:
```
def visualize_knn(classifier,X_test,N=5, K=7):
  # This visualization routine makes use of GLOBAL Train_Images and Test_Images␣
 ↪variables
  # to visualize the K nearest neighbors of the first N Test Images

  dist = classifier.compute_distances_no_loops(X_test[:N,:])
  _, k_idxs = classifier.predict_labels(dist,k=K)
  k_idxs  = np.vstack(k_idxs)
  testim  = montage(Test_Images[:N,:],grid_shape=(N,1),channel_axis=3)
  trainim = montage(Train_Images[k_idxs.ravel(),:
 ↪],grid_shape=(N,K),channel_axis=3)
  plt.imshow(np.concatenate((testim,trainim),axis=1))
  plt.axis('off');
  plt.title('Test [leftmost column], K_neighbors [right columns]');

visualize_knn(classifier,X_test)
```




#### 4.0.2 Normalizing image descriptors:

Let us try normalizing each image here by subtracting by its mean and scaling to have unit norm.

[ ]:
```
# Normalize each image descriptor to have zero-mean and unit-length

X_train_norm = X_train
X_test_norm  = X_test

X_train_centered = X_train-np.mean(X_train, axis=1, keepdims=True)
X_train_norm = X_train_centered/np.linalg.norm(X_train_centered, axis=1,␣
 ↪keepdims=True)

X_test_centered = X_test-np.mean(X_test, axis=1, keepdims=True)
X_test_norm = X_test_centered/np.linalg.norm(X_test_centered, axis=1,␣
 ↪keepdims=True)

print('Train data shape: ', X_train_norm.shape)
print('Test data shape: ', X_test_norm.shape)
```

```
Train data shape:  (9000, 3072)
Test data shape:  (3856, 3072)
```

We calculate the accuracies again using k = 1 and k = 3 and see that the accuracies are much better compared to those we obtained without any preprocessing on the images!

```
[ ]: classifier = KNearestNeighbor()
classifier.train(X_train_norm,y_train)

# Classify using the efficient no_loops implementation
dists = classifier.compute_distances_no_loops(X_test_norm)
y_test_pred, k_labels = classifier.predict_labels(dists, k=3)

# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 1323 / 3856 correct => accuracy: 0.343102
```

Written question: Normalization produces image descriptors that have unit length. Prove that minimizing the euclidean distance of such descriptors is equivalent to maximizing the cosine similarity. Here is an example of latex in markdown that might be helpful: $||x-y||^2 = x^Tx - 2x^Ty + y^Ty$

Proof: Say the image descriptors are represented by 2 vectors $u, v$ with unit length. Its euclidean distance is

$$||u-v||_2^2 = u^\top u - 2u^\top v + v^\top v = 2 - 2u^\top v$$

Minimizing this value indicates that we need to minimize the term $u^\top v$. Yet in the cosine similarity,

$$\cos(u,v) = \frac{u^\top v}{||u||||v||} = u^\top v$$

Finally,

$$||u-v||_2^2 = 2 - 2\cos(u,v)$$

implying that minimizing the Euclidean distance is equivalent to maximizing the cosine similarity.

### 4.1 KNN with HOG

The previous parts all directly used raw pixels from input images to compute distances with k-NN. In this part, we will first use the Histogram of Oriented Gradients (HOG) as features for each image. We will use these features with our kNN implementation to find the nearest neighbours. Please read the descriptions and fill in the functions below.

```
[ ]: def compute_gradient(image):
    """
      Computes the gradients in both x and y directions.
      Computes the magnitudes of the gradients.
      Computes the angles from the gradients and map to range [0, 180 deg].
      NOTE: You may *NOT* use np.gradient
      Inputs:
      - image: A (32,32) numpy array corresponding to a grayscale image
               or a (32,32,3) array corresponding to a color image
      Returns:
      - magnitudes: A numpy array of shape (32, 32) where magnitudes[i, j]
        is the magnitude of the gradient at the (i, j) pixel in the input image.
```

```
    - angles: A numpy array of shape (32, 32) where angles[i, j]
      is the angle of the gradient at the (i, j) pixel in the input image.
    HINT: First write thefunction assuming a grayscale input and get a final␣
↪accuracy. Then write the color version.
          You may wish to use numpy.take_along_axis()
    """
  from scipy import signal
  if image.ndim==2:
    image=image[:,:,np.newaxis]
  height,width,colors=image.shape
  img_rows=np.zeros((height,width,colors))
  img_cols=np.zeros((height,width,colors))
  for color in range(colors):
    kernel_rows=np.array([[-1,0,1]])
    img_rows[:,:,color]=signal.convolve2d(image[:,:,color], kernel_rows,␣
↪boundary='symm', mode='same')

    kernel_cols=np.array([[-1,0,1]]).T
    img_cols[:,:,color]=signal.convolve2d(image[:,:,color], kernel_cols,␣
↪boundary='symm', mode='same')

  magnitude=np.sqrt(img_rows**2+img_cols**2)

  theta=np.arctan2(img_cols, img_rows)
  angle=np.degrees(theta)%180

  idx = np.argmax(magnitude, axis=2)[:, :, np.newaxis]
  magnitudes = np.take_along_axis(magnitude, idx, axis=2).squeeze(axis=2)
  angles = np.take_along_axis(angle, idx, axis=2).squeeze(axis=2)
  return magnitudes, angles
```

[ ]:
```
def bin_gradient(angles, magnitudes, n_orient, pixels_per_cell):
  """
  Given the gradient orientations and magnitudes of an image, creates
  a histogram of orientations weighted by gradient magnitudes
  Inputs:
  - angles: A numpy array of shape (32, 32) where angles[i,j]
      is the angle of the gradient at the (i,j) pixel in the input image.
  - magnitudes: A numpy array of shape (32, 32) where magnitudes[i,j]
      is the magnitude of the gradient at the (i,j) pixel in the input image.
  - n_orient: An int representing the number of orientations to bin in histogram
  - pixels_per_cell: An int representing the number of rows/columns of pixels
      in each spatial cell
  Returns:
  - oriented_histogram: A numpy array of shape (32/4=8, 32/4=8,9)
      for pixels_per_cell=4 and n_orient=9
   """
```

```
 n_y,n_x = angles.shape
 oriented_histogram = np.zeros((int(n_y//pixels_per_cell),int(n_x//
↪pixels_per_cell),n_orient))
 angle_unit=180/n_orient
 orient_center=[0]*n_orient
 for cell_y in range(int(n_y//pixels_per_cell)):
   for cell_x in range(int(n_x//pixels_per_cell)):
     for j in range(cell_y * pixels_per_cell, (cell_y + 1) * pixels_per_cell):
       for i in range(cell_x * pixels_per_cell, (cell_x + 1) *␣
↪pixels_per_cell):
         angle=int(angles[i,j]/angle_unit)
         oriented_histogram[cell_y, cell_x, angle] += magnitudes[i, j]
 return oriented_histogram
```

**NOTE :** Once we create a histogram based on the gradient of the image we need to normalize it. Gradients of an image are sensitive to overall lighting. If you make the image darker by dividing all pixel values by 2, the gradient magnitude will change by half, and therefore the histogram values will change by half.

Ideally, we want our image features to be independent of lighting variations. In other words, we would like to "normalize" the histogram so they are not affected by lighting variations.

We have provided the normalization code below.

[ ]:
```
def block_normalize(oriented_histogram, cells_per_block, clip = True,␣
↪epsilon=1e-5):
 """
 Normalizes the histogram in blocks of size cells_per_block.
 Inputs:
 - oriented_histogram: A numpy array of shape (num_cell_rows, num_cell_cols,␣
↪num_orient)
   representing the histogram of oriented gradients of the input image.
 - cells_per_block: An int representing the number of rows/columns of cells␣
↪that
   should together be normalized in the same block (you can assume )
 - clip: If true, this clips the normalized descriptor of each block to ensure␣
↪that no values are larger than .2 (and then
   renormalizes to ensure the clipped descriptor is unit-norm), just as SIFT␣
↪does
 - epsilon: A float indicating the small amount added to the denominator when
   normalizing to avoid dividing by zero.
 Returns:
 - normalized_blocks: A numpy array of shape (num_cell_rows-cells_per_block+1,␣
↪num_cell_cols+cells_per_block+1,
     cells_per_block,cells_per_block,num_orient) where normalized_blocks[i,j]␣
↪is a normalized [cells_per_block,cells_per_block,num_orient] "SIFT"␣
↪descriptor
```

```
    """

    n_blocks_y = oriented_histogram.shape[0]-cells_per_block+1
    n_blocks_x = oriented_histogram.shape[1]-cells_per_block+1
    normalized_blocks = np.
    ↪zeros((n_blocks_y,n_blocks_x,cells_per_block,cells_per_block,oriented_histogram.
    ↪shape[2]))

    for i in range(n_blocks_y):
      for j in range(n_blocks_x):
          block = oriented_histogram[i:i+cells_per_block, j:j+cells_per_block, :]

          block = block / np.sqrt(np.sum(block**2) + epsilon**2)

          if clip:
              block = np.minimum(block, 0.2)
              block = block / np.sqrt(np.sum(block**2) + epsilon**2)

          normalized_blocks[i, j] = block

    return normalized_blocks
```

After implementing your HOG functions, please run the cells below to test the results. You should expect to get an accuracy slightly higher than that with unnormalized raw pixels.

[ ]:
```
def compute_hog(image,n_orient=9,pixels_per_cell=4,cells_per_block=4):
    """
    Builds a Histogram of Oriented Gradients (HOG) weighted by gradient magnitudes
    from an input image
    Inputs:
    - image: A numpy array of shape (32, 32) containing one grayscaled image.
    Outputs:
    - histogram: A 1D numpy array that represents the HOG descriptor for the␣
    ↪image.
    """
    assert(image.dtype == 'float64')
    # Read in image and convert to grayscale
    # if len(image.shape) > 2:
    #  image = np.mean(image,2)

    # Compute gradient
    magnitudes, angles = compute_gradient(image)

    # Bin gradients into cells
    oriented_histogram = bin_gradient(angles, magnitudes, n_orient,␣
    ↪pixels_per_cell)
```

```
  # Block normalize the cells
  normalized_blocks = block_normalize(oriented_histogram, cells_per_block)

  # Return flattened descriptor (without making an additional copy)
  return normalized_blocks.ravel()
```

[ ]:
```
# Check out HOG descriptor for a single image
#image = X_train[0].mean(2) # Initially, build representation for grayscale image
image = X_train[0].reshape(img_size,img_size,3);
plt.figure(figsize=(14,8))
plt.subplot(1,3,1)
plt.imshow(image.astype('uint8'));
plt.axis('off')
plt.title('Input Image')

pixels_per_cell=4
cells_per_block=4
n_orient=9
angle_step = 180 // n_orient

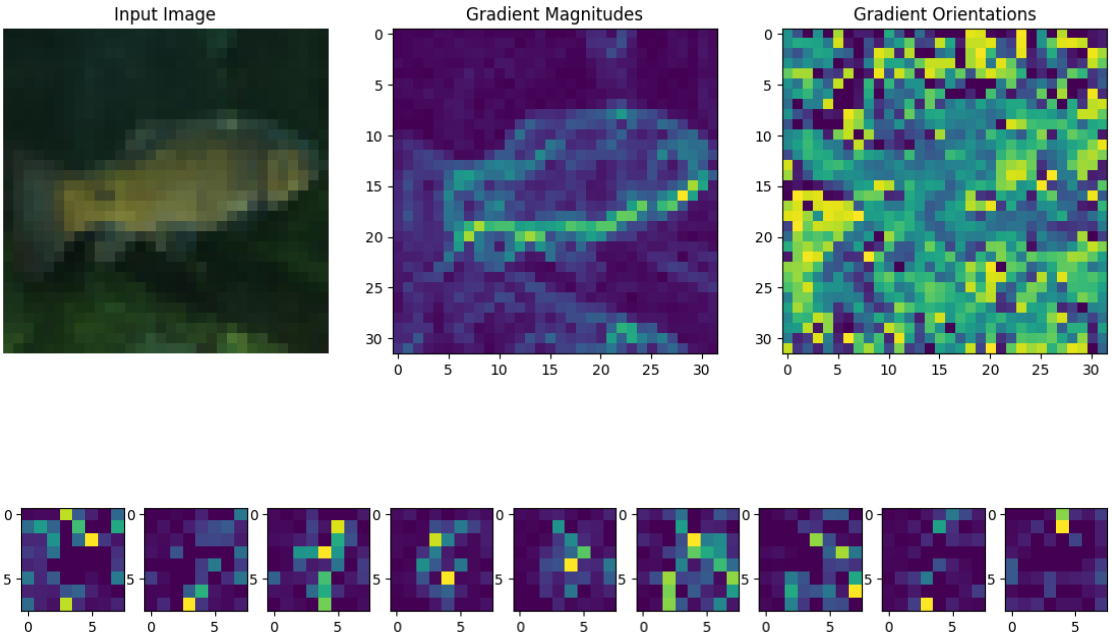
# Step 1: compute gradients
magnitudes, angles = compute_gradient(image)

plt.subplot(1,3,2)
plt.imshow(magnitudes)
plt.title('Gradient Magnitudes')

plt.subplot(1,3,3)
plt.imshow(angles)
plt.title('Gradient Orientations')


# Step 2: Bin gradients into cells
oriented_histogram = bin_gradient(angles, magnitudes, n_orient, pixels_per_cell)
plt.figure(figsize=(14,8))
#plt.suptitle('Oriented Histograms')
for i in range(n_orient):
  plt.subplot(1,n_orient,i+1)
  plt.imshow(oriented_histogram[:,:,i])

# Step 3: Block normalize the cells
normalized_blocks = block_normalize(oriented_histogram, cells_per_block)
```




This part will take some time to run for the full dataset. Approx 1 to 2mins.

[ ]:
```
X_train_hog = np.array([compute_hog(X_train[i].reshape(img_size,img_size,3))␣
 ↪for i in range(num_train)])
X_test_hog  = np.array([compute_hog( X_test[i].reshape(img_size,img_size,3))␣
 ↪for i in range(num_test)])
print('Train data shape: ', X_train_hog.shape)
print('Test data shape: ' , X_test_hog.shape)
```

```
Train data shape:  (9000, 3600)
Test data shape:  (3856, 3600)
```

[ ]:
```
classifier = KNearestNeighbor()
classifier.train(X_train_hog,y_train)
dists = classifier.compute_distances_no_loops(X_test_hog)
```

[ ]:
```
# Compute and print the fraction of correctly predicted examples
y_test_pred, k_labels  = classifier.predict_labels(dists, k=3)
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 1643 / 3856 correct => accuracy: 0.426089
```

You can also visualize the K nearest images for some randomly selected examples from the test set using the k_idxs list you returned in predict_labels trained with HOG descriptors.

[ ]:
```
visualize_knn(classifier,X_test_hog)
```




### 4.2 Extra credit 1: parameter tweaking

Add in descriptions of your optimal parameter settings and the resulting performance, compared to your default parameter settings and your default performance

[ ]:
```
pixels_per_cell = 8
cells_per_block = 4
n_orient = 9

X_train_hog_64 = np.array([
    compute_hog(
        X_train[i].reshape(img_size, img_size, 3),
        n_orient=n_orient,
        pixels_per_cell=pixels_per_cell,
        cells_per_block=cells_per_block
    )
    for i in range(num_train)
])

X_test_hog_64 = np.array([
    compute_hog(
        X_test[i].reshape(img_size, img_size, 3),
        n_orient=n_orient,
        pixels_per_cell=pixels_per_cell,
        cells_per_block=cells_per_block
```

```
    )
    for i in range(num_test)
])

classifier_64 = KNearestNeighbor()
classifier_64.train(X_train_hog_64, y_train)

dists_64 = classifier_64.compute_distances_no_loops(X_test_hog_64)

y_test_pred_64, _ = classifier_64.predict_labels(dists_64, k=3)

num_correct = np.sum(y_test_pred_64 == y_test)
accuracy_64 = float(num_correct) / num_test

print("Got %d / %d correct => accuracy: %f"% (num_correct, num_test,␣
 ↪accuracy_64))

print("HOG descriptor dimension:", X_train_hog_64.shape[1])
```

```
Got 1544 / 3856 correct => accuracy: 0.400415
HOG descriptor dimension: 144
```

### 4.3 Extra credit 2: low-rank descriptors

[ ]:
```
def low_rank_svd(X_train_desc, X_test_desc, rank):
  mean_desc = np.mean(X_train_desc, axis=0, keepdims=True)
  X_train_centered = X_train_desc - mean_desc
  X_test_centered = X_test_desc - mean_desc
  U, S, Vt = np.linalg.svd(X_train_centered, full_matrices=False)
  V_low = Vt[:rank].T
  X_train_low = X_train_centered @ V_low
  X_test_low = X_test_centered @ V_low
  return X_train_low, X_test_low


def evaluate_low_rank(name, X_train, X_test, reduction=3, k=3):
    original_dim = X_train.shape[1]
    rank = max(1, original_dim // reduction)

    print("\n" + "=" * 50)
    print(name)
    print("=" * 50)

    print("Original descriptor size:", original_dim)
    print("Low-rank descriptor size:", rank)
    print("Reduction factor: %.2fx" % (original_dim / rank))
```

```
    Xtrain_low, Xtest_low = low_rank_svd(X_train, X_test, rank)

    classifier_low = KNearestNeighbor()
    classifier_low.train(Xtrain_low, y_train)

    dists_low = classifier_low.compute_distances_no_loops(Xtest_low)
    y_pred_low, _ = classifier_low.predict_labels(dists_low, k=k)

    num_correct = np.sum(y_pred_low == y_test)
    accuracy = float(num_correct) / num_test

    print("Accuracy with k=%d: %.4f" % (k, accuracy))

    return {
        "descriptor": name,
        "original_dim": original_dim,
        "low_rank_dim": rank,
        "reduction": original_dim / rank,
        "accuracy": accuracy
    }

raw_result = evaluate_low_rank("Raw pixel␣
 ↪descriptors",X_train,X_test,reduction=3,k=3)

X_train_centered_norm = (X_train - np.mean(X_train, axis=1, keepdims=True))
X_test_centered_norm = (X_test - np.mean(X_test, axis=1, keepdims=True))

X_train_norm_svd = X_train_centered_norm / (np.linalg.
 ↪norm(X_train_centered_norm, axis=1, keepdims=True) + 1e-12)
X_test_norm_svd = X_test_centered_norm / (np.linalg.norm(X_test_centered_norm,␣
 ↪axis=1, keepdims=True) + 1e-12)

norm_result = evaluate_low_rank("Normalized pixel descriptors",
    X_train_norm_svd,
    X_test_norm_svd,
    reduction=3,
    k=3
)

hog_result = evaluate_low_rank(
    "HOG descriptors",
    X_train_hog,
    X_test_hog,
    reduction=3,
    k=3
)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_877/2095172655.py in <cell line: 0>()
     43     }
     44
---> 45 raw_result = evaluate_low_rank("Raw pixel␣
 ↪descriptors",X_train,X_test,reduction=3,k=3)
     46
     47 X_train_centered_norm = (X_train - np.mean(X_train, axis=1,␣
 ↪keepdims=True))

NameError: name 'X_train' is not defined
```

### 4.4 Extra credit 3: Modify your existing code to use deep features

[ ]:
```
# ===== your code here! =====

# ===== end of code =====
```

### 4.5 AI Assistance

1. Did you use any artificial intelligence assistance in solving any part of this assignment?

Answer here: Yes

2. If you answered "Yes", give full details (e.g. "I used Claude to identify and debug errors in my code implementation for Question 2")

Answer here: Claude was used to optimize the code and check for the details and errors in my code.